

AdvR 04 – Control Flow

Choices

if / else Basics

```
if (condition) true_action  
if (condition) true_action else false_action
```

Typically with compound statements:

```
grade <- function(x) {  
  if (x > 90) "A"  
  else if (x > 80) "B"  
  else if (x > 50) "C"  
  else "F"  
}  
grade(85)
```

```
#> [1] "B"
```

if **returns a value** — you can assign the result:

```
x1 <- if (TRUE) 1 else 2  
x2 <- if (FALSE) 1 else 2  
c(x1, x2)
```

```
#> [1] 1 2
```

if Returns NULL Without else

Without else, if invisibly returns NULL when FALSE. Since `c()` and `paste0()` drop NULL inputs, this enables a compact idiom:

```
greet <- function(name, birthday = FALSE) {  
  paste0("Hi ", name,  
    if (birthday) " and HAPPY BIRTHDAY")  
}  
greet("Jaime", TRUE)
```

```
#> [1] "Hi Jaime and HAPPY BIRTHDAY"
```

```
greet("Maria", FALSE)
```

```
#> [1] "Hi Maria"
```

Invalid Inputs

The condition must be a **single** TRUE or FALSE:

```
if ("x") 1
```

```
#> Error in if ("x") 1: argument is not interpretable as logical
```

```
if (logical()) 1
```

```
#> Error in if (logical()) 1: argument is of length zero
```

```
if (NA) 1
```

```
#> Error in if (NA) 1: missing value where TRUE/FALSE needed
```

A vector of length > 1 also errors (as of R 4.2):

```
if (c(TRUE, FALSE)) 1
```

```
#> Error in if (c(TRUE, FALSE)) 1: the condition has length > 1
```

Vectorised if: ifelse() and case_when()

if works with scalars. For **vectors**, use `ifelse()`:

```
x <- 1:10  
ifelse(x %% 2 == 0, "even", "odd")
```

```
#> [1] "odd"  "even" "odd"  "even" "odd"  "even" "odd"  
#> [8] "even" "odd"  "even"
```

Note: NA in test propagates to output. Best when yes/no are the same type.

For **multiple conditions**, use `dplyr::case_when()`:

```
dplyr::case_when(  
  x %% 35 == 0 ~ "fizz buzz",  
  x %% 5 == 0 ~ "fizz",  
  x %% 7 == 0 ~ "buzz",  
  .default = as.character(x)  
)
```

```
#> [1] "1"    "2"    "3"    "4"    "fizz" "6"    "buzz"  
#> [8] "8"    "9"    "fizz"
```

switch() Statement

Compact alternative to chained if/else if:

```
x_option <- function(x) {  
  switch(x,  
    a = "option 1",  
    b = "option 2",  
    c = "option 3",  
    stop("Invalid `x` value")  
  )  
}  
x_option("b")
```

```
#> [1] "option 2"
```

Always include a default (last unnamed argument) — otherwise unmatched inputs silently return NULL.

switch() Fall-Through

Empty RHS passes to the next match (like C's switch):

```
legs <- function(x) {  
  switch(x,  
    cow = , horse = , dog = 4,  
    human = , chicken = 2,  
    plant = 0,  
    stop("Unknown input"))  
}  
legs("cow")
```

```
#> [1] 4
```

cow, horse, and dog all fall through to 4.

Use switch() only with **character** inputs (numeric x is error-prone).

Exercises (Choices)

❶ What type of vector does each call to `ifelse()` return?

```
ifelse(TRUE, 1, "no")  
ifelse(FALSE, 1, "no")  
ifelse(NA, 1, "no")
```

Read the documentation and write down the rules in your own words.

❷ Why does the following code work?

```
x <- 1:10  
if (length(x)) "not empty" else "empty"  
  
x <- numeric()  
if (length(x)) "not empty" else "empty"
```

Solutions (Choices)

- ❶ `ifelse()` returns the type of the selected value:

```
ifelse(TRUE, 1, "no")    # double (from yes)
```

```
#> [1] 1
```

```
ifelse(FALSE, 1, "no")   # character (from no)
```

```
#> [1] "no"
```

```
ifelse(NA, 1, "no")      # logical NA
```

```
#> [1] NA
```

Rule: returns entry from yes when TRUE, from no when FALSE, NA when NA. Output type comes from whichever branch is selected.

- ❷ `if()` accepts numeric input: 0 is treated as FALSE, all other numbers as TRUE. `length(1:10)` is 10 (truthy); `length(numeric())` is 0 (falsy).

Loops

for Loop Basics

```
for (item in vector) perform_action
```

```
for (i in 1:3) {  
  print(i)  
}
```

```
#> [1] 1
```

```
#> [1] 2
```

```
#> [1] 3
```

for assigns item to the **current environment**, overwriting existing variables:

```
i <- 100  
for (i in 1:3) {}  
i # 3, not 100
```

```
#> [1] 3
```

next and break

Two ways to exit early:

- **next** — skip current iteration
- **break** — exit entire loop

```
for (i in 1:10) {  
  if (i < 3) next  
  print(i)  
  if (i >= 5) break  
}
```

```
#> [1] 3
```

```
#> [1] 4
```

```
#> [1] 5
```

1. Preallocate output — otherwise $O(n^2)$:

```
means <- c(1, 50, 20)
out <- vector("list", length(means))
for (i in seq_along(means)) {
  out[[i]] <- rnorm(10, means[[i]])
}
```

2. Never use `1:length(x)` — fails when `x` has length 0:

```
1:length(numeric()) # c(1, 0) --- wrong!
```

```
#> [1] 1 0
```

```
seq_along(numeric()) # integer(0) --- correct
```

```
#> integer(0)
```

3. Loops strip S3 attributes from dates and factors:

```
xs <- as.Date(c("2020-01-01", "2010-01-01"))  
for (x in xs) print(x)    # prints raw doubles!
```

```
#> [1] 18262
```

```
#> [1] 14610
```

Fix: index with `[[` instead:

```
for (i in seq_along(xs)) print(xs[[i]])
```

```
#> [1] "2020-01-01"
```

```
#> [1] "2010-01-01"
```

while and repeat

Use when the number of iterations is **unknown**:

- `while(condition) action` — runs while condition is TRUE
- `repeat action` — runs forever until `break`

```
# Flip coins until heads
flips <- 0
while (sample(c("H", "T"), 1) != "H") {
  flips <- flips + 1
}
```

Flexibility hierarchy: `for` $\subset$ `while` $\subset$ `repeat`.

Use the **least flexible** tool that works. In practice, prefer `for` (or `map()`/`apply()`) for data analysis.

Exercises (Loops)

❶ Why does this code succeed without errors or warnings?

```
x <- numeric()
out <- vector("list", length(x))
for (i in 1:length(x)) {
  out[i] <- x[i] ^ 2
}
out
```

❷ When this code is evaluated, what can you say about the vector being iterated?

```
xs <- c(1, 2, 3)
for (x in xs) {
  xs <- c(xs, x * 2)
}
xs
```

❸ What does this code tell you about when the index is updated?

```
for (i in 1:3) {
  i <- i * 2
  print(i)
}
```

- ❶ `1:length(x)` creates `c(1, 0)` when `x` is length 0. Iteration 1: `x[1]` returns `NA` (out-of-bounds), assigned to `out[1]`. Iteration 2: `x[0]` returns `numeric(0)` (zero-index), assigned to `out[0]` (no-op). No error because each step is valid R — just not what you intended. Use `seq_along()` instead.

Solutions (Loops) — 2–3

- ② `for` evaluates the vector **once** at the start. Modifying `xs` inside the loop doesn't change the iteration. The loop runs 3 times (original values 1, 2, 3):

```
xs <- c(1, 2, 3)
for (x in xs) xs <- c(xs, x * 2)
xs
```

```
#> [1] 1 2 3 2 4 6
```

- ③ The index is **reset at the start** of each iteration. Reassigning `i` inside the loop doesn't affect the next iteration:

```
for (i in 1:3) {
  i <- i * 2
  print(i)
}
```

```
#> [1] 2
```

```
#> [1] 4
```

```
#> [1] 6
```

Prints 2, 4, 6 — not 2, 8, 32.